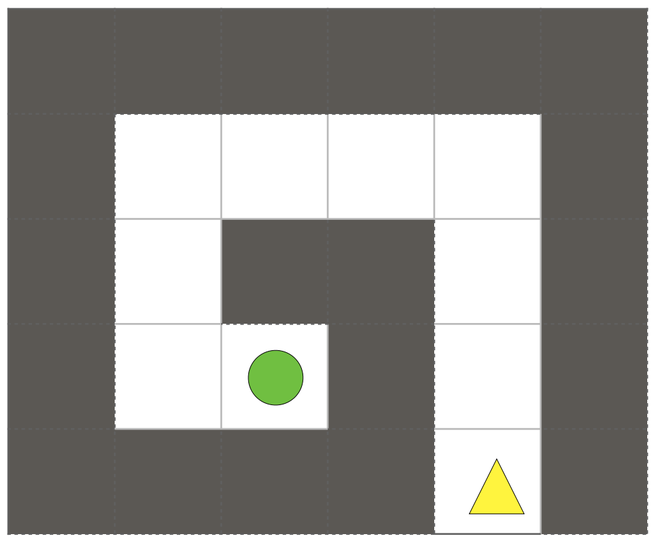
20. Computational Power

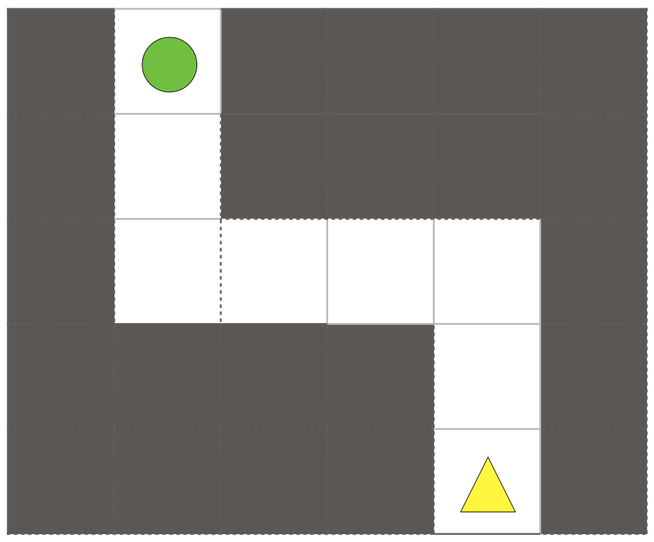
Differences in Computational Power: CCs vs SCs

Spiral



CC computable.
SC computable.

Zig-Zag



Not CC computable.
SC computable.

Differences in Power

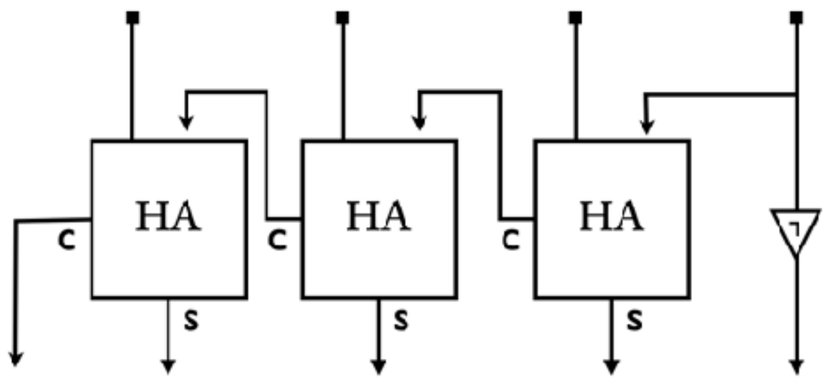
Zig-Zag is not CC computable because it requires different responses to the same input at different times. But CCs are history-independent.

Zig-Zag is SC computable, because SC memory allows the machine to enter into different input-output states over time, depending on its history.

The Memory Moral

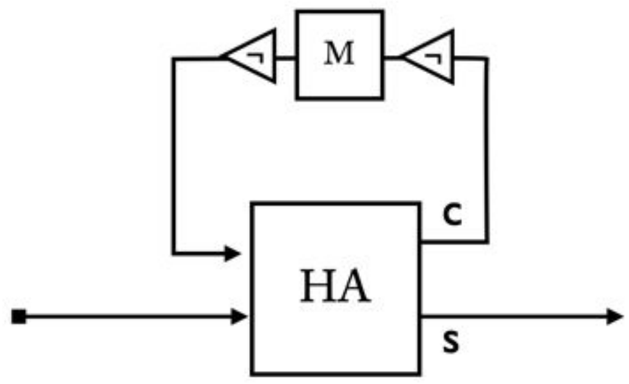
Memory allows a machine to display flexible responses to inputs. As a consequence, memory increases computational power.

Binary +1 [0-15]



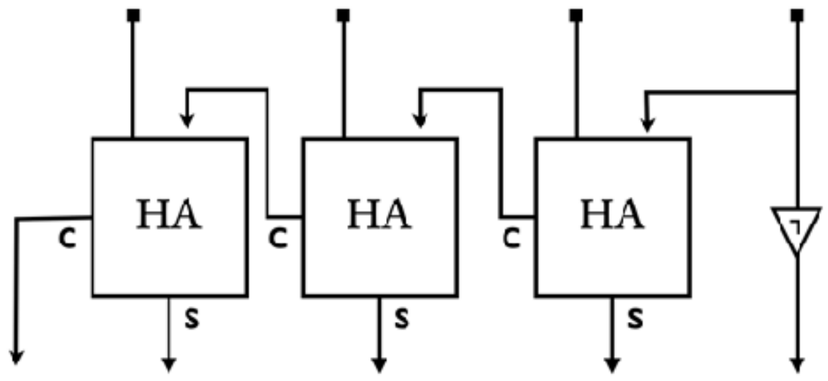
CC computable.
SC computable.

Binary +1



Not CC computable.
SC computable.

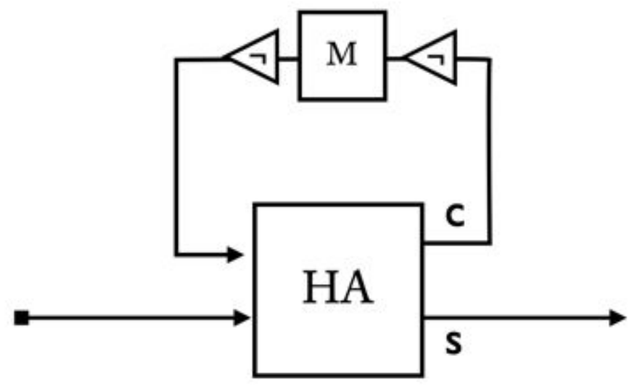
CC Powers and Limitations



CC Binary +1 [0-15]

- CCs have no memory or history responsiveness: they always respond in the same way to the same input.
- The input size for any given CC is fixed by its internal mechanisms.
- CCs can compute any bounded function.
- CCs compute only bounded functions.
- Practically: the higher the bound, the bigger the machine.

SC Powers and Limitations



SC Binary +1

- SCs contain memory and are history responsive: they can respond in different ways to the same input.
- The input size for any given SC is the infinite (or extendable) dimension of time.
- SCs can compute any bounded function.
- SCs compute some unbounded functions.
- Practically: for many functions, the higher the bound, the machine stays the same. (For other functions, this isn't true.)

"Infinite ends, finite means."

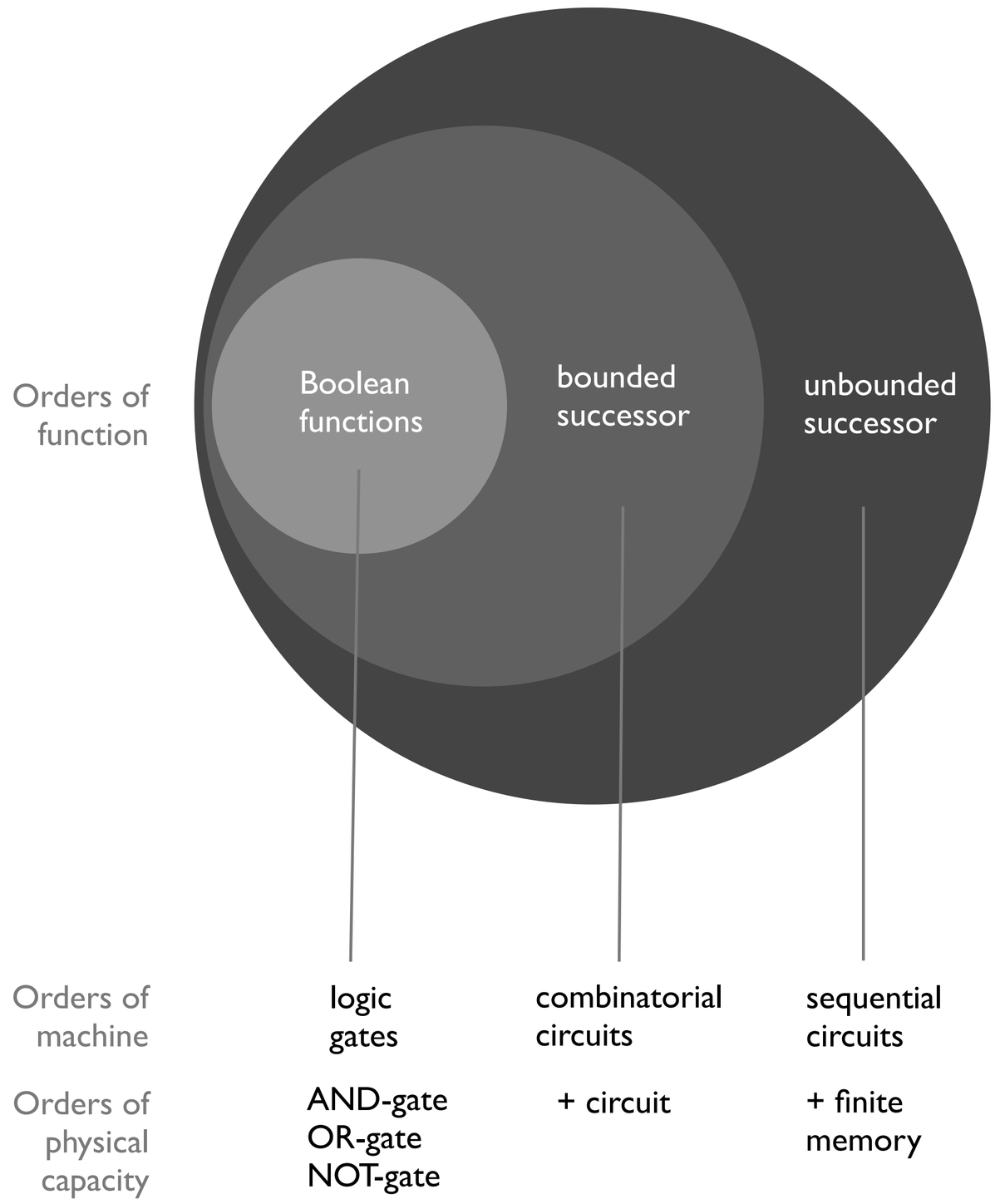
CC Implementation

- Fast.
- Fixed input size.
- The larger the problem, the larger the machine.
- Good for problems with fixed size.
- Compare: perception.

SC Implementation

- Slow.
- Unbounded input size.
- For many problems, the machine stays the same size.
- Good for problems with unpredictable size.
- Compare: thought.

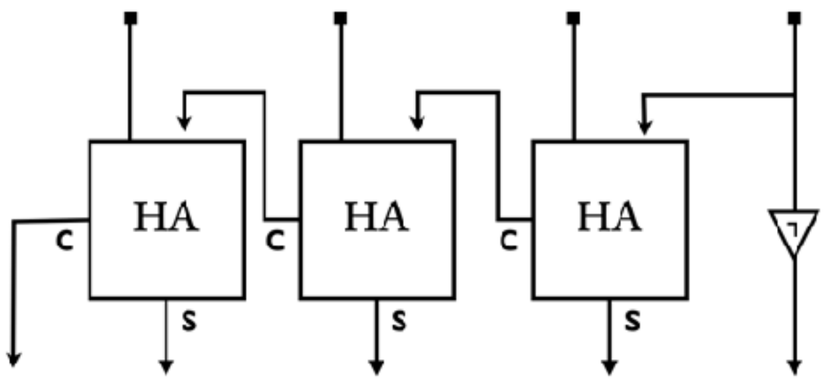
Orders of Computational Power



Definition: Computational Power
A kind of machine M1 has greater computational power than a kind of machine M2 iff

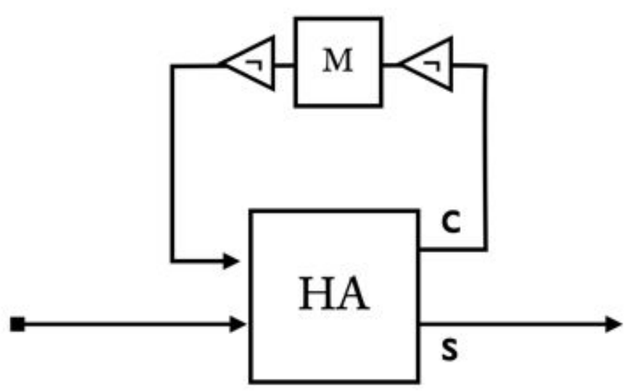
(a) for any function *f* and any system of representation *R*:
if some M2 can compute *f* under *R*, then some M1 can compute *f* under *R*

(b) there is some function *f* and system *R* such that:
some M1 can compute *f* under *R*, but no M2 can compute *f* under *R*.



Binary +1 [0-15]

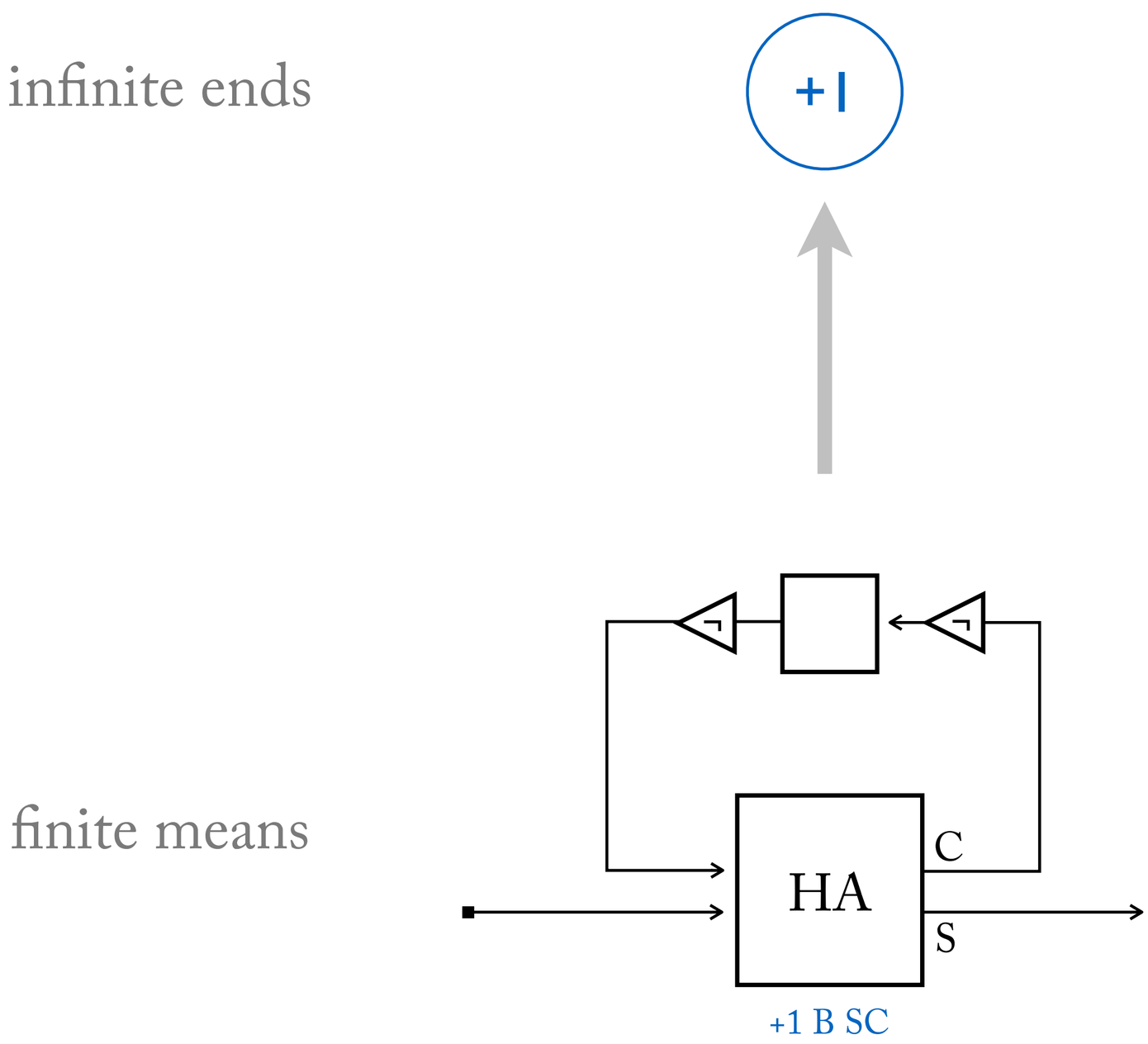
CC computable.
SC computable.



Binary +1

Not CC computable.
SC computable.

> For language is quite peculiarly confronted by an unending and truly boundless domain, the essence of all that can be thought. It must therefore make infinite employment of finite means, and is able to do so through the power which produces identity of language and thought.
>
> — Humboldt, *On Language*, 1836



What allows SCs to overcome their own finite physical limitations? They exploit the potential infinity of time.

Memory isn't just necessary for achieving infinite potential. It seems to be necessary for *coherence* too.

Language. I'd like to buy a carne asada taco.

Argument. I think, therefore I am.

A Map of Power and Abstraction

Different classes of function are computed by machines with different physical features. The more functions computable by a kind of machine, the greater its *computational power*.

